******Docker Full Notes and Commands******

What is OS?

Operating System a type of system software. It basically manages all the resources of the computer.
An operating system acts as an interface between the User and computer hardware.

What is Virtualization?

Virtualization is a technology that allows you to create and run multiple virtual instances of computing resources, such as operating systems, servers, or storage, on a single physical machine.

What is Containerization?

Containerization is a technology that allows you to package an application and all of its dependencies (like libraries, configurations, and binaries) into a container.

Key Benefits of Containerization:-

Isolation: - Each container is isolated from others, so applications in separate containers can't interfere with each other.

Portability: - Containers can run on any system that supports the container runtime (Docker, Docker for example), whether it's your local machine, a cloud server, or on-premises hardware.

Scalability: - Containers make it easy to scale applications by simply adding or removing containers.

Efficiency: - Containers are more lightweight than traditional virtual machines because they share the host system's kernel instead of running their own OS.

What is Container?

Container is combination of OS, Software, libraries and Configuration files

Containers are lightweight a single server or (Virtual Machine) VM can run several containers simultaneously. It implements a high-level API to provide these lightweight containers that run processes in isolation

Popular Container Technologies:-

Docker: The most widely used containerization platform.

Podman: A daemonless alternative to Docker.

Containers vs. Virtual Machines (VMs):

Feature	Container	Virtual Machine
OS	Shares host OS kernel	Has its own guest OS
Size	Lightweight	Heavy (includes full OS image)
Startup Time	Seconds	Minutes
Resource Usage	Efficient	More resource-intensive
Isolation	Process-level isolation	Full isolation

What is Podman? (Short for Pod Manager)

Red Hat **Podman** is an open-source container management tool developed by **Red Hat.**

It is designed to simplify the creation, management, and deployment of containers and containerized applications. Podman is a lightweight, daemon less container engine that offers functionality similar to Docker.

What is pod?

A Pod is a logical host for one or more containers. Pods are used to run applications in containers.

What is Container file?

A container image is a static file that contains executable code and all the dependencies required to create a container on a computing system.

What is Registry?

A **registry** is a storage location for information related to software or systems.

Container Registry: - A storage and distribution system for container images.

Examples: -

- Docker Hub

- Amazon Elastic Container (AEC)

Registry (ECR) Registry URL: - docker.io , quay.io, registry.access.redhat.com, registry.redhat.io etc.

==========

**How to Install Docker on Redhat Linux (RHEL) **

URL:- https://docs.docker.com/engine/install/rhel/

Remove Podman & Buildah

#yum remove -y podman buidah

Set up the repository

Install the dnf-plugins-core package (which provides the commands to manage your DNF repositories) and set up the repository.

dnf install dnf-plugins-core -y
dnf config-manager --add-repo
https://download.docker.com/linux/rhel/docker-ce.repo

Install Docker Engine

Install the Docker packages.

sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y

Start Docker Engine

#systemctl start docker

Enable docker service
#systemctl enable --now docker

To check status of docker service

#systemctl status docker

#docker --version {to check docker version}

==========

How to Install Docker on Amazon Linux

Install Docker Engine

#yum install -y docker

Start Docker Engine

#systemctl start docker

Enable docker service

#systemctl enable --now docker

To check status of docker service

#systemctl status docker

#docker --version {to check docker version}

==========

Pull, Create Start, Stop, Restart, Rename & Copy Command in Container

docker search almalinux (Search registry for image)
docker pull almalinux (Pull an image from a registry)
docker images (to show download images)

(OR)
docker image ls

docker create -it almalinux { Create but do not start a container}
docker start <container_name> {to start the container}

docker attach <container_name> {to connect container exit (exit the container but stop container)

ctrl+p+q (without stop exit)

#docker ps {to show running container}
#docker ps -a {to show both start and stop container}

Where, ps=process status

-a= all

#docker restart <container_name> (to restart the container)

docker rename <old name container> <new name container> (to rename the container)

Example:-

docker rename crazy_wilson c1 (to rename the container)

Where,
crazy_wilson=old name of container
c1=new name of container

docker inspect c1 (to show the configuration of a container)
docker top c1 (to show he running processes of a container)

docker exec c1 touch file1 (to create file in a running container)

docker exec c1 ls (to list in a running container)

docker cp /etc/fstab c1:/tmp (to copy files to running container)

docker exec c1 ls /tmp (list)

docker cp ./. c1:/tmp (to copy all files from current location to container)

docker cp c1:/file1 /tmp

(Copy a directory on a container to a directory on the host)
docker cp containerA:/myapp containerB:/newapp (Copy a directory on a container into a directory on another)
docker exec c1 rm /tmp/fstab (to delete file in container)

docker exec c1 ls /tmp (list)

docker run -itd --name node1 centos:centos7.9.2009 (image download and create container)

docker logs c1 (to show container logs)

docker stats (show a live stream of container resource usage statistics)

To Remove Container

docker stop c1 (to stop the container)
docker rm c1 (to remove stop container)

docker rm --force c1 (to remove running container)

To Remove Image

docker rmi docker.io/library/httpd (to remove only image)

docker rm --force docker.io/library/almalinux (to remove image with container)

docker image rm --all (to remove all images)

[NOTE: Container Configuration file : /etc/containers/registries.conf]

Port Mapping with Apache Server

Port mapping in Docker is a technique that allows you to access

services running inside a Docker container from outside the container

docker run -d -p <HostPort:containerport> <imagename>

docker run -itd --name webserver -p 8080:80 httpd (map port to container)

(i= interactive, t = terminal, d=detach mode, p=port)

docker port webserver (to show Mapping port)

docker port --all (to show all Mapping Ports)

echo "**Welcome to Docker Server**" > index.html (to create index.html file)

docker cp index.html webserver:/usr/local/apache2/htdocs (copy

index.html files)

docker exec webserver ls /usr/local/apache2/htdocs (list data)

firefox http://serverip:8080 (to check /Host Machine IP Address)

Mariadb Service

docker run -d --name mariadb-server -e MARIADB_ROOT_PASSWORD=redhat -p 8181:3306 mariadb

docker ps

yum install mysql -y

mysql -h 192.168.0.165 -P 8181 -u root -p (access mariadb)

Password: redhat

Login done
exit

Managing a Container Network

The chapter provides information about how to communicate Containers.

In Docker, there are two networks

1. Rootless networking - the container does not have an IP address.
2. Rootful networking - the container has an IP address.

docker network ls

How to Connect two containers

docker network create mynet (to create network)

docker run -itd --name con1 almalinux (to create container con1)

docker run -itd --name con2 almalinux (to create container con2)

docker ps (to show container)

docker network connect mynet con1 (to connect network to con1 container)

docker network connect mynet con2 (to connect network to con2 container)

docker network inspect mynet (to show network info)

OR

docker inspect con1 | grep -iwA 10 networks

Testing

docker exec -it con1 bash

[root@36e715907efd /]# ping -c 3 10.89.0.3

docker exec -it con2 bash

[root@36e715907efd /]# ping -c 3 10.89.0.2

exit

[Disconnect the container named con1 from the network named mynet]

docker network disconnect mynet con1

docker network disconnect mynet con1

docker network inspect mynet (to verify)

How to Delete Network

docker network rm mynet (to delete networks)

docker network prune (Remove all unused networks)

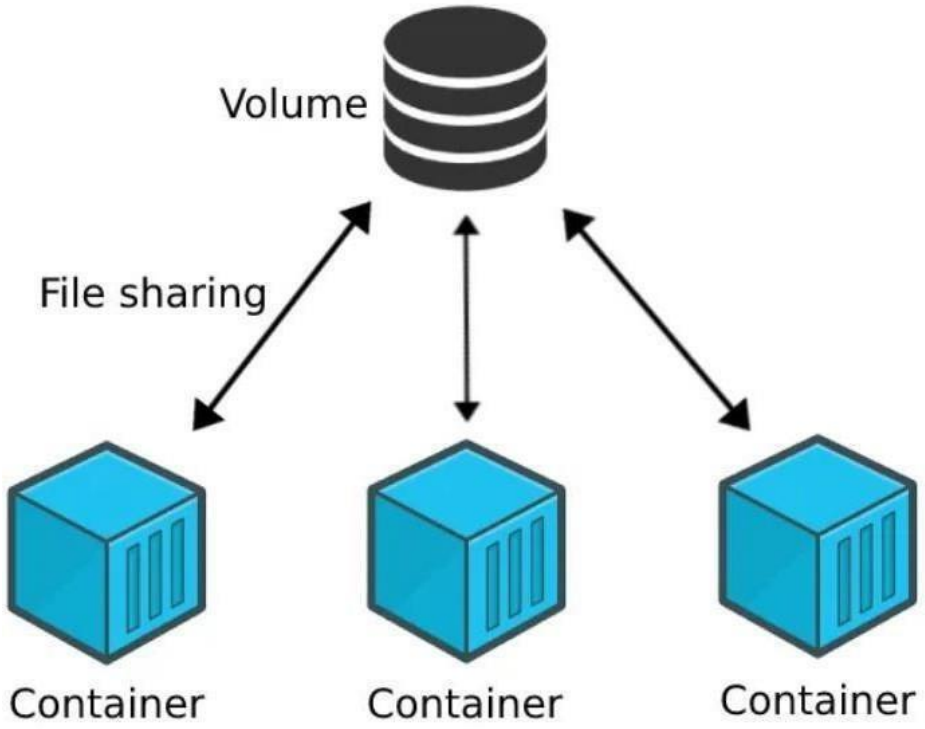
docker network ls

==========

Volumes (Persisting data)

Volumes are persistent data stores for containers, created and managed by Docker

Mountpoint Path: /var/lib/containers/storage/volumes/vol1/_data



Host to Container Volume Sharing

mkdir /opt/files

touch /opt/files/test{1..3}.txt

docker run -it --name hostcon -v /opt/files:/incoming --privileged=true ubuntu /bin/bash

root@7b0673c18b27:/# ls

root@7b0673c18b27:/# cd incoming/

root@7b0673c18b27:/incoming# ls test1.txt

test2.txt test3.txt (to show files)

root@7b0673c18b27:/incoming# touch test{4..6}.txt (to create files)

root@7b0673c18b27:/incoming# ls

root@7b0673c18b27:/incoming

exit

cd /opt/files

ls (to show files)

Create Volume in container and how to share volume another container

docker run -it --name con1 -v /volume1 ubuntu /bin/bash (create container with folder)

root@cb12886d880f:/# ls (list)

root@cb12886d880f:/# cd volume1/ (go to folder)

root@cb12886d880f:/volume1# touch test{1..5}.html (to create files) [Open New Terminal or Tab]

docker run -it --name con2 --privileged=true --volumes-from con1 ubuntu /bin/bash (create other container name is con2)

root@45647df0f955:/# cd volume1

root@45647df0f955:/volume1# ls

test1.html test2.html test3.html test4.html test5.html

root@45647df0f955:/volume1# touch update{1..5}.txt

root@45647df0f955:/volume1# ls

[Goto 1st Terminal or Tab]

[Testing]

root@cb12886d880f:/# cd volume1

root@cb12886d880f:/volume1# ls

test1.html test2.html test3.html test4.html test5.html update1.txt update2.txt update3.txt update4.txt update5.txt

root@cb12886d880f:/volume1# exit

DONE

root@cb12886d880f:/volume1# ls

test1.html test2.html test3.html test4.html test5.html update1.txt update2.txt update3.txt update4.txt update5.txt

root@cb12886d880f:/volume1# exit

DONE

Create Custom images

docker run -it --name mycontainer ubuntu /bin/bash

touch /tmp/testfile

exit

docker diff mycontainer (to check different)

docker commit mycontainer updateimage (to create image)

docker images

Now create container from this image

docker run -it --name newcontainer updateimage /bin/bash

ls

cd /tmp

ls (to show testfile)
exit

How to push image on your docker Account

docker login docker.io
Username:
Password:

docker push f7d8bafbd9a9 docker.io/redhatwala/httpd-test

Go to docker hub site, login your account and check

EXPORTING AND IMPORTING CONTAINERS (backup and restore)

you can use the docker export command to export a current snapshot of your running container into a tarball on your local machine.

You can use the docker import command to import a tarball (revert back to it later) and save it as a filesystem image.

Prerequisites: The container-tools meta-package is installed. #

docker run -dt --name myubi rhel9 (to create container) #

docker ps -a (to show container)

docker attach myubi (to attach)

[root@1b03f2f8540f /]# echo "**hello**" > testfile

(to create testfile) Detach from the container with CTRL+p and CTRL+q (exit)

[Export the file system of the myubi as a myubi-container.tar on the local machine:]

docker export -o myubi-container.tar 1b03f2f8540f (export the filesytem)

ls -l [List]

docker import myubi-container.tar myubi-imported (to import)

docker images

docker run -it --name myubi-imported myubi-imported /bin/bash

ls

cat testfile (read file)

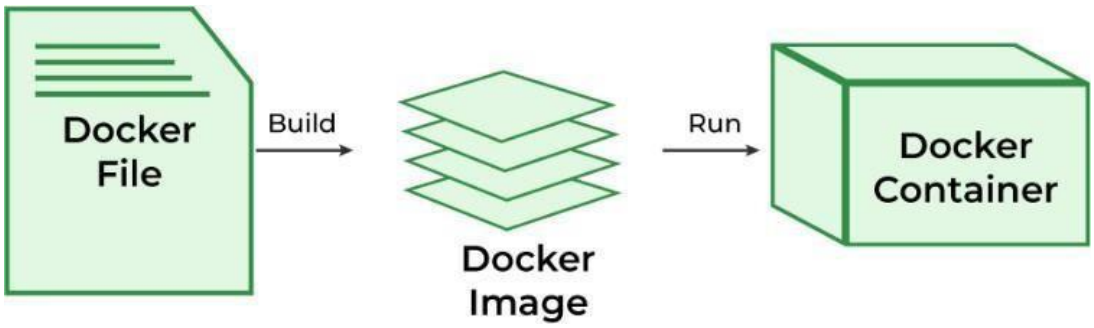
CTRL+p and CTRL+q (exit)

DONE

Container File/Docker File

A Container file is a text file that contains instructions for creating a container image.

Automation of Docker image Creation



Container File Options:

FROM: for base image. This command must be on top of the Containerfile/Dockerfile

RUN: to execute commands, it will create a layer in image

MAINTAINER: Author/Owner/Description

COPY: Copy files from local machine we need to provide source, destination. (Only local system)

ADD: Similar to COPY but, is provides a feature to download files from internet, also we extract tarfile at docker image side.

EXPOSE: to expose ports such as port 8080 for Apache. WORKDIR: to set working directory for a container CMD: execute commands but during container creation

ENTRYPOINT: similar to CMD, but has higher priority over CMD, 1st commands will be executed by ENTRYPOINT only

ENV: Environment Variables

This example uses the http base image, copies a custom index.html file, and exposes port 80

vim Dockerfile

```
FROM httpd:2.4    [Use an official base image]

MAINTAINER Dinesh Patel    [Set an Author for the container]

COPY ./index.html /usr/local/apache2/htdocs/  [Copy the website
content into the container]

EXPOSE 80   [Expose port 80]

CMD ["httpd-foreground"] [Start the Apache server]


:wq!        {Forcefully save and  quit}
```

vim index.html
Welcome to Docker Class

:wq! {Forcefully save and quit}

docker build -t httpd . {to create image}

#docker images {to check docker images}

docker run -d --name webserver1 -p 808:80 httpd {to create container}

curl http://<server-ip>

Also, check the browser.

Complete Docker Topics and Commands

*****Thank You!*****